

REPUBLIC OF YEMEN

Ministry of Higher Education of Sciences

TAIZ UNIVERSITY

Faculty Of Engineering IT

SE Department



الجمهورية اليمنية

وزارة التعليم العالي والبحث العلمي

جامعة تعز

كلية الهندسة وتقنية المعلومات

قسم البرمجيات

# **Design and Development of a Unified Gesture Framework for Cross–Platform Mobile Applications**

**A Final Year Project Report**

Submitted in partial fulfillment of the requirements for the Degree of  
Bachelor of Science in Software Engineering

**Prepared by:**

- Ziad Faisal Gazem
- Gamal Sadek Saeed
- Hafeed Shaheed
- Moharram Al-jaradi

**Supervisor**

Dr. Ziad Al-Robih

**College of Engineering and Information Technology**

**Taiz University**

**Taiz, Yemen**

# الإقرار

نحن، الموقعون أدناه، طلاب كلية الهندسة وتقنية المعلومات في جامعة تعز، نقر بأن مشروع التخرج المعنون ``مكتبة إيماءات اللمس المتقدمة لتطبيقات React " Native هو عملنا الأصلي.

وحسب علمنا، فإن هذا المستند لا يحتوي على أي مواد سبق نشرها أو كتابتها من قبل شخص آخر، ولا مواد تم قبولها للحصول على أي درجة أو دبلوم آخر في هذه الجامعة أو أي مؤسسة تعليمية أخرى، باستثناء ما تمت الإشارة إليه في النص. ونقر باستخدام أطر العمل مفتوحة المصدر والمراجع التقنية وفقاً لتراخيصها، والتي تم توثيقها بالكامل داخل التقرير.

# صفحة الاعتماد

تم فحص واعتماد مشروع التخرج هذا المعنون ``مكتبة إيماءات اللمس المتقدمة لتطبيقات React Native" كجزء من متطلبات الحصول على درجة البكالوريوس في هندسة البرمجيات.

مشرف المشروع:

---

د. زياد الربيع

التاريخ: \_\_\_\_\_

الممتحن الداخلي:

---

الاسم: \_\_\_\_\_

التاريخ: \_\_\_\_\_

رئيس القسم:

---

الاسم: \_\_\_\_\_

التاريخ: \_\_\_\_\_

## الإهداء

نهدي هذا المشروع إلى عائلتنا، التي لولا دعمها اللامحدود، وصبرها، وتشجيعها لما أمكننا إتمام مسيرتنا الأكاديمية. كما نهدي هذا العمل إلى أساتذتنا وإلى مجتمع البرمجيات مفتوحة المصدر، الذين ألهمتنا أدواتهم الأساسية ومعارفهم المشتركة. وأخيراً، نهدي هذا العمل إلى وطننا الحبيب اليمن، آمليين أن نساهم في تقدمه التكنولوجي ومستقبله المشرق.

## شكر وتقدير

أولاً وقبل كل شيء، نشكر الله عز وجل على منحنا القوة والصحة والحكمة لإتمام هذا المشروع بنجاح. نود أن نعرب عن أعمق امتناننا لمشرف مشروعنا، د. زياد الربيع، لتوجيهاته القيمة، وملاحظاته البناءة، ودعمه المستمر طوال فترة تطوير هذا البحث. كما نتقدم بخالص الشكر والتقدير لأعضاء هيئة التدريس في كلية الهندسة وتقنية المعلومات بجامعة تعز لتزويدنا بالمعرفة الأساسية اللازمة للقيام بمشروع بهذا الحجم التقني.

# الملخص

مع تطور تطبيقات الهواتف الذكية وتزايد تعقيد واجهات المستخدم، أصبحت إيماءات اللمس عنصراً حيوياً لتوفير تجربة مستخدم سلسة وتفاعلية. يواجه مطورو التطبيقات متعددة المنصات (مثل React Native) تحديات مستمرة تتمثل في تأخر الاستجابة الملحوظ الناتج عن تمرير الأحداث عبر الجسر (The Bridge) بين الواجهة الأصلية للنظام وبيئة JavaScript. يهدف هذا المشروع إلى تصميم وتطوير إطار عمل موحد ومتقدم (Gesture Framework) مخصص لمعالجة إيماءات اللمس بمختلف مستوياتها، بحيث يتجاوز هذه القيود المعمارية ويضمن أداءً فائقاً بمعدل 60 إطاراً في الثانية.

يُقدم هذا الإطار منظومة شاملة تتكون من خمس حزم أساسية تغطي طيفاً واسعاً من التفاعلات: بدءاً من الإيماءات الأساسية والمنفصلة (كالضغط المزدوجة والضغط المطول)، مروراً بإيماءات السحب والانزلاق المتصلة (Drag/Pan)، وصولاً إلى الإيماءات متعددة الأصابع والتحويلات الهندسية المعقدة (مثل إيماءة القرص للتكبير والتدوير المتزامن). ويعتمد التصميم المعماري للمشروع بشكل جوهري على نقل معالجة الإيماءات بالكامل إلى مسار الواجهة الأصلية (UI Thread) بالاعتماد على مكتبة react-native-gesture-handler والمتغيرات المشتركة في Reanimated، مما يلغي الاختناقات المرتبطة بالجسر. بالإضافة إلى ذلك، يوسع الإطار قدراته ليدمج بيانات الحساسات (مثل مستشعر التقارب Proximity) ومحرك رسم عالي الأداء لتنفيذ تطبيقات التوقيع الرقمي والتفاعلات المتقدمة.

أظهرت التقييمات المعمارية نجاح هذا النهج في تحقيق تفاعل شبه فوري وتوفير استقرار مرجعي عالٍ. ومن خلال تقديم واجهة برمجة تطبيقات (API) نظيفة ومستقرة عبر خطافات (Hooks) مخصصة، يوفر الإطار حلاً جذرياً يُسهّل على مجتمع المطورين دمج التفاعلات الحركية المعقدة في تطبيقاتهم، مما يعزز من جودة تجربة المستخدم عبر مختلف المنصات دون المساس بسرعة الاستجابة.

# ABSTRACT

As smartphones evolve and user interfaces grow increasingly complex, touch gestures have become a vital component for delivering seamless and interactive user experiences. Cross-platform application developers (such as those using React Native) face persistent challenges regarding noticeable response latency caused by event serialization across “The Bridge” between the native platform and the JavaScript environment. This project aims to design and develop a unified, advanced Gesture Framework tailored for processing touch gestures across all complexities, bypassing architectural constraints to guarantee high performance at a consistent 60 frames per second (fps).

The framework introduces a comprehensive ecosystem comprising five primary packages that cover a wide spectrum of interactions. These range from discrete, basic touch events (e.g., double tap, long press), to continuous swipe and drag interactions (Drag/Pan), up to multi-finger combinations and complex geometric transformations (such as simultaneous pinch-to-zoom and rotation). The core architectural design of the project relies on migrating gesture processing entirely to the Native UI Thread. By leveraging react-native-gesture-handler alongside shared values from Reanimated, the framework eliminates bridge-related bottlenecks. Furthermore, the architecture extends its capabilities to integrate hardware sensor data (e.g., proximity sensors) and a high-performance drawing engine tailored for digital signature applications and advanced continuous interactions.

Architectural evaluations demonstrate the success of this approach in achieving near-instantaneous interaction and providing high referential stability. By delivering a clean and stable Application Programming Interface (API) through custom React Hooks, the framework offers a radical solution that facilitates the integration of complex kinematic interactions into the broader developer community’s applications. This ultimately enhances the quality of user experiences across different platforms without compromising responsiveness.

# List of Tables

|     |                                                    |    |
|-----|----------------------------------------------------|----|
| 1.1 | تدعمها التي والإيماءات gesture-kit حزم             | 5  |
| 3.1 | الإيماءات على التعرف خوارزميات مقارنة              | 14 |
| 3.2 | الإيماءات من فئة لكل المستهدفة الأداء معايير       | 14 |
| 4.1 | والحركة السحب إيماءات مقارنة                       | 19 |
| 4.2 | gesture-kit-sensor حزمة إيماءات                    | 27 |
| 4.3 | gesture-kit وPanResponder بين الاستجابة زمن مقارنة | 31 |



# List of Figures

# Contents

|    |                                                                |
|----|----------------------------------------------------------------|
| 2  | 1 المقدمة                                                      |
| 3  | 1.1 خلفية الدراسة                                              |
| 3  | 2.1 مشكلة البحث                                                |
| 4  | 3.1 أهداف المشروع                                              |
| 4  | 4.1 هيكل المكتبة --- نظرة عامة                                 |
| 4  | 5.1 أهمية الدراسة                                              |
| 5  | 6.1 نطاق الدراسة وحدودها                                       |
| 6  | 2 الإطار النظري والدراسات السابقة                              |
| 7  | 1.2 أنظمة معالجة الإيماءات في تطبيقات الهواتف المحمولة         |
| 7  | 1.1.2 تطور أطر معالجة الإيماءات                                |
| 7  | 2.2 تصنيف الإيماءات اللمسية وخصائصها التقنية                   |
| 8  | 1.2.2 المعمارية الداخلية لأنظمة التعرف على الإيماءات           |
| 8  | 3.2 التحديات التقنية في تطوير مكتبات الإيماءات متعددة المنصات  |
| 8  | 1.3.2 مشكلة تأخر الجسر والحلول الحديثة                         |
| 9  | 2.3.2 إدارة تضارب الإيماءات                                    |
| 9  | 4.2 مكتبات الإيماءات والأنماط المعمارية للمكتبات مفتوحة المصدر |
| 9  | 1.4.2 مبادئ تصميم المكتبات القابلة لإعادة الاستخدام            |
| 9  | 2.4.2 نمط الخطافات في منظومة React                             |
| 9  | 5.2 نظم الحساسات الداخلية في الأجهزة المحمولة                  |
| 10 | 6.2 خلاصة الإطار النظري                                        |
| 11 | 3 المنهجية وبنية النظام                                        |
| 12 | 1.3 منهجية البحث والتصميم                                      |
| 12 | 1.1.3 تحليل المتطلبات                                          |
| 12 | 2.3 تصميم معمارية النظام المتكاملة                             |

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| 3.3      | اختيار الخوارزميات وتكاملها . . . . .                       | 13        |
| 1.3.3    | خوارزميات التعلم الآلي الأساسية . . . . .                   | 13        |
| 2.3.3    | توليد المرشحين . . . . .                                    | 14        |
| 4.3      | جمع البيانات، السياق اللغوي، ومواد تدريب النموذج . . . . .  | 14        |
| 5.3      | تدفق النظام ومسار التوصية . . . . .                         | 14        |
| 1.5.3    | استيعاب البيانات وبنية القياس عن بعد . . . . .              | 14        |
| 2.5.3    | مسار التدريب غير المتصل (التحليل العواملي الكامن) . . . . . | 15        |
| 3.5.3    | مسار الاسترجاع المتصل . . . . .                             | 15        |
| <b>4</b> | <b>التنفيذ والنتائج</b>                                     | <b>16</b> |
| 1.4      | نظرة عامة على هيكل التنفيذ . . . . .                        | 17        |
| 2.4      | الحزمة الأولى: إيماءات اللمس الأساسي . . . . .              | 17        |
| 1.2.4    | المفهوم والبنية . . . . .                                   | 17        |
| 2.2.4    | مثال تطبيقي: إيماءة الضغط الواحدة . . . . .                 | 17        |
| 3.2.4    | مثال تطبيقي: الضغطة المزدوجة والضغط المطول . . . . .        | 18        |
| 3.4      | الحزمة الثانية: إيماءات السحب والحركة . . . . .             | 19        |
| 1.3.4    | المفهوم والتصنيف . . . . .                                  | 19        |
| 2.3.4    | مثال تطبيقي: بطاقة قابلة للسحب . . . . .                    | 20        |
| 4.4      | الحزمة الثالثة: الإيماءات متعددة الأصابع . . . . .          | 21        |
| 1.4.4    | الأسس الرياضية . . . . .                                    | 21        |
| 2.4.4    | مثال تطبيقي: تكبير الصور بإيماءة القرص . . . . .            | 22        |
| 5.4      | الحزمة الرابعة: إيماءات حساس التقارب . . . . .              | 23        |
| 1.5.4    | مبدأ عمل حساس التقارب . . . . .                             | 23        |
| 2.5.4    | نظام الأنواع لحزمة التقارب . . . . .                        | 23        |
| 3.5.4    | مثال تطبيقي: إيقاظ الشاشة بالتقارب . . . . .                | 24        |
| 4.5.4    | مثال تطبيقي: الضغطة الهوائية بدون لمس . . . . .             | 25        |
| 6.4      | الحزمة الخامسة: إيماءات الحساسات الحركية . . . . .          | 26        |
| 1.6.4    | الحساسات المدعومة والأنواع المعرّفة . . . . .               | 26        |
| 2.6.4    | الإيماءات الحركية المدعومة . . . . .                        | 27        |
| 3.6.4    | مثال تطبيقي: إيماءة الهز لتحديث البيانات . . . . .          | 27        |

|    |       |                                              |
|----|-------|----------------------------------------------|
| 28 | 4.6.4 | مثال تطبيقي: إمالة الجهاز للتحكم في الكاميرا |
| 29 | 5.6.4 | مثال تطبيقي: دمج التقارب والحساسات           |
| 31 | 7.4   | تقييم الأداء والمقارنة                       |
| 31 | 1.7.4 | نتائج قياسات الأداء                          |
| 31 | 2.7.4 | مناقشة النتائج                               |

## 5 الخاتمة والتوصيات 33

|    |     |                              |
|----|-----|------------------------------|
| 34 | 1.5 | ملخص النتائج                 |
| 34 | 2.5 | الخاتمة                      |
| 35 | 3.5 | التوصيات والأعمال المستقبلية |

## Bibliography

36

## قائمة الاختصارات

|              |                                   |
|--------------|-----------------------------------|
| <b>UI/UX</b> | User Interface / User Experience  |
| <b>HAL</b>   | Hardware Abstraction Layer        |
| <b>API</b>   | Application Programming Interface |
| <b>NPM</b>   | Node Package Manager              |

# مسرد المصطلحات

**واجهة برمجة التطبيقات (API)** مجموعة من القواعد والبروتوكولات التي تسمح لتطبيقات البرامج المختلفة بالتواصل مع بعضها البعض.

**الجسر (The Bridge)** آلية في Native React تسمح لبيئة JavaScript بالتواصل مع الواجهات الأصلية للنظام لمعالجة الإيماءات.

**الشاشة السعوية (Capacitive Screen)** تقنية شاشات اللمس الحديثة التي تعتمد على الخصائص الكهربائية لجسم الإنسان لاستشعار اللمس المتعدد.

**واجهة المستخدم (UI) وتجربة المستخدم (UX)** واجهة المستخدم تشير إلى العناصر المرئية للتطبيق، بينما تشير تجربة المستخدم إلى الانسيابية وسرعة الاستجابة أثناء تفاعل المستخدم مع النظام.

# 1 CHAPTER

المقدمة

## 1.1 خلفية الدراسة

شهد مجال تطوير تطبيقات الهواتف الذكية تطوراً متسارعاً في السنوات الأخيرة، لا سيما مع ظهور أطر العمل متعددة المنصات التي أتاحت للمطورين إمكانية بناء تطبيقات متكاملة تعمل على كلٍ من نظامي التشغيل iOS و Android بقاعدة شفرة مصدرية موحدة. في طليعة هذه الأطر يأتي React Native، الذي وصفه Dabit [dabit2019] بأنه إطار يُتيح بناء تطبيقات أصلية (Native Applications) حقيقية، لا تطبيقات ويب مُغلّفة، وذلك عبر جسر تواصل (The Bridge) يربط بيئة JavaScript بواجهات برمجة النظام الأصلية مباشرةً.

ومع تزايد الاعتماد على الأجهزة اللمسية وتطور واجهات المستخدم التفاعلية، باتت إيماءات اللمس (Touch Gestures) عنصراً محورياً في تصميم تجربة المستخدم الحديثة. ويُعدّ التعامل مع هذه الإيماءات مهمةً دقيقةً ومعقدةً من الناحية التقنية؛ إذ تتطلب الاستجابة الفورية لأنماط متعددة كالسحب (Pan)، والتكبير (Pinch)، والتدوير (Rotation)، والضغط المطوّل (Long Press)، فضلاً عن إيماءات الحساسات كالهز (Shake) والإمالة (Tilt) والتقارب (Proximity). وقد أكد Masiello وFriedmann [masiello2017] أن تحقيق سلاسة التفاعل عند معدل 60 إطاراً في الثانية (60fps) يستلزم نقل معالجة الإيماءات الحرجة إلى طبقة الواجهة الأصلية (Native Layer) متى أمكن ذلك.

ويتمثل التحدي الأكبر في هذا السياق في بناء حل مرّن وموثوق وقابل لإعادة الاستخدام يمكن توزيعه على مجتمع المطورين عبر مستودعات الحزم العامة كnpm. وقد أكد Casciaro وMammino [casciaro2020] أن المكتبة الجيدة يجب أن تلتزم بمبدأ المسؤولية الواحدة (Single Responsibility Principle)، وتُعلن صراحةً عن التبعية المطلوبة عبر آلية peerDependencies لتجنب تعارض الإصدارات.

## 2.1 مشكلة البحث

على الرغم من توفر بعض المكتبات في بيئة React Native للتعامل مع إيماءات اللمس، إلا أنها تفتقر في الغالب إلى ثلاثة عناصر جوهرية: أولها التكامل المباشر مع بيانات مستشعرات الجهاز (كحساس التقارب Proximity Sensor ومقياس التسارع Accelerometer) في آنٍ واحد مع إيماءات الشاشة اللمسية. وثانيها واجهة برمجية (API) موحدة تجمع كافة فئات الإيماءات تحت مظلة واحدة متسقة. وثالثها توثيق احترافي وهيكلي نشر سليم يستوفي معايير المجتمع المفتوح المصدر.

يصف Dabit [dabit2019] نظام الاستجابة الافتراضي لـ React Native وهو PanResponder بأنه قادر على تتبع حركات اللمس الأساسية، غير أنه يُنفذ معالجته بالكامل على مسار JavaScript، مما يجعله عُرضةً



للتأثر بضغط الحسابات وانخفاض معدل الإطارات. ويزيد من تعقيد هذه المشكلة غيابُ دعم إيماءات الحساسات الفيزيائية كالهز والإمالة والتقارب ضمن أي إطار موحد.

### 3.1 أهداف المشروع

يهدف هذا المشروع إلى:

1. تصميم وبناء حزم JavaScript مخصصة لبيئة React Native تُعنى بمعالجة إيماءات اللمس وربطها بمعطيات المستشعرات الحركية في الوقت الفعلي.
2. توفير واجهات برمجية (Hooks & Components) سهلة الاستخدام للتعامل مع كامل طيف الإيماءات: من الضغطة البسيطة حتى التحويلات الهندسية المزدوجة وإيماءات الحساسات.
3. دمج بيانات المستشعرات --- مقياس التسارع والجيروسكوب وحساس التقارب --- مع أحداث اللمس لتقديم استجابة سياقية أكثر دقةً وذكاءً.
4. نشر المكتبة على منصة npm وفق المعايير المهنية، بما يشمل التوثيق الكامل وإعداد التبعيات وإصدار دلالي منضبط.
5. تحقيق أداء مستقر عند 60 إطاراً في الثانية عبر نقل المعالجة إلى UI Thread مباشرةً.

### 4.1 هيكل المكتبة --- نظرة عامة

تتكوّن gesture-kit من سبع حزم رئيسية موزعة على مستودعين (Monorepo):

### 5.1 أهمية الدراسة

تُسهم هذه المكتبة في ردم الفجوة التقنية بين واجهات تفاعل المستخدم ومعطيات مستشعرات الجهاز. تتجلى أهميتها في محورين رئيسيين:

أولاً --- الصعيد التقني: أثبت Masiello وFriedmann [masiello2017] أن دمج بيانات الحساسات مع الإيماءات يفتح آفاقاً واسعة لتطبيقات الواقع المعزز والألعاب التفاعلية وتطبيقات إمكانية الوصول (Accessibility). وبتوفير gesture-kit لدعم حساس التقارب وإيماءات الهز والإمالة، تُصبح تجارب التفاعل اللائمية (Touchless)

جدول 1.1: حزم gesture-kit والإيماءات التي تدعمها

| اسم الحزمة               | الفئة         | الإيماءات الرئيسية            |
|--------------------------|---------------|-------------------------------|
| gesture-kit-basic-touch  | لمس أساسي     | Tap, Double Tap, Long Press   |
| gesture-kit-drag-pan     | حركة وسحب     | Pan, Drag, Swipe, Fling       |
| gesture-kit-multi-finger | متعدد الأصابع | Pinch, Spread, Multi-Swipe    |
| gesture-kit-transform    | تحويلات       | Rotation, Scale, Combined     |
| gesture-kit-sequences    | تسلسلات       | Tap&Hold, Complex Chains      |
| gesture-kit-proximity    | تقارب         | HandNear, Hover, ProximityTap |
| gesture-kit-sensor       | حساسات        | Shake, Tilt, Flip, FreeFall   |

(Interactions) في متناول أي مطور .

ثانياً --- **صعيد المجتمع التقني**: يُشكّل نشرها مفتوح المصدر على npm مرجعاً قياسياً يُتبنّى في مشاريع تجارية وأكاديمية. ويؤكد Casciaro و Mammino [casciaro2020] أن التصميم الجيد للمكتبات المفتوحة المصدر يُعزز ثقافة المشاركة ويُقلص تكاليف التطوير في مشاريع لا تُحصى.

## 6.1 نطاق الدراسة وحدودها

يقتصر نطاق هذا المشروع على بيئة React Native مع التركيز على الإيماءات اللمسية وبيانات المستشعرات المدمجة في الهاتف. أما حدود المشروع فهي:

- لا يتناول المشروع معالجة إشارات المستشعرات الخارجية (كأجهزة Bluetooth أو الإكسسوارات الخارجية).
- لا يشمل دعم أطر عمل تطوير التطبيقات الأخرى كFlutter أو Ionic.
- يقتصر اختبار الأداء على الأجهزة المتاحة في بيئة التطوير.
- لا يتناول المشروع معالجة بيانات المستشعرات على مستوى الخادم.

## 2 CHAPTER

### الإطار النظري والدراسات السابقة

## 1.2 أنظمة معالجة الإيماءات في تطبيقات الهواتف المحمولة

شهد مجال تطوير تطبيقات الهواتف الذكية تطوراً متسارعاً في التعامل مع إيماءات اللمس، إذ انتقل من نماذج بدائية تعتمد على الأحداث المنفردة إلى أطر عمل متكاملة قادرة على التعرف على أنماط حركية معقدة في الوقت الفعلي. تُعدّ إيماءات اللمس (Touch Gestures) اليوم العمود الفقري لتجربة المستخدم في أي تطبيق محمول، إذ تُشير الدراسات إلى أن ما يزيد على 70% من تفاعلات المستخدم مع تطبيقات الهواتف تتم عبر إيماءات لمسية متنوعة.

يُقدم Dabit [dabit2019] في كتابه React Native in Action أحد أشمل المراجع التقنية في هذا المجال، إذ يُفصّل آلية عمل نظام الاستجابة لللمس (Gesture Responder System) في React Native. ويُبيّن أن الإطار الافتراضي المُقدّم (PanResponder) يوفر واجهة للتعامل مع أحداث اللمس الأساسية، غير أن هذا النظام يعاني من قيد بنيوي جوهري يتمثل في تشغيله الكامل داخل مسار JavaScript، مما يُعرّضه للتأثر بأي ضغط حسابي على هذا المسار. وقد أفرز هذا القيد حاجة ملحة لأطر عمل أكثر كفاءة تُجري المعالجة على المسار الأصلي للواجهة مباشرة.

### 1.1.2 تطور أطر معالجة الإيماءات

تتسم الأجيال الأولى من أطر الإيماءات في React Native بمحدودية واضحة؛ فقد كانت تعتمد بشكل حصري على الجسر (The Bridge) لنقل أحداث اللمس من الطبقة الأصلية إلى بيئة JavaScript. وكما يُوضح Dabit [dabit2019]، فإن هذا الجسر يُسبب تأخيراً كامناً (Latency) في استجابة الإيماءات بسبب التسلسل (Serialization) والإرسال (Dispatch) غير المتزامن للأحداث عبر حدود الخيوط (Thread Boundaries). في المقابل، تُرسي مكتبة react-native-gesture-handler نهجاً مغايراً جذرياً، إذ تُنفّذ منطق التعرف على الإيماءات بالكامل داخل المسار الأصلي للواجهة (UI Thread)، مما يُلغي تأخير الجسر ويضمن استجابة آنية تعكس حركة الإصبع بدقة متناهية. هذا التحول المعماري يُمثّل نقلة نوعية في تصميم مكتبات الإيماءات للمنصات المتقاطعة.

## 2.2 تصنيف الإيماءات اللمسية وخصائصها التقنية

تنقسم الإيماءات اللمسية في سياق التطبيقات المحمولة إلى فئتين رئيسيتين تتباينان في طبيعة الأحداث التي تُولّدانها: أولاً: الإيماءات المنفصلة (Discrete Gestures) وهي إيماءات تُنتج حدثاً واحداً محدداً اكتمال شروطه،

وتشمل الضغطة الواحدة (Tap)، والضغطة المزدوجة (Double Tap)، والضغط المطول (Long Press). تعتمد آلية التعرف عليها أساساً على عاملي الزمن والمسافة؛ إذ تُعرّف الضغطة الواحدة بأنها لمس لا تتجاوز مدته 500 ميلي ثانية مع تحقيق قيد المسافة (أقل من 20 بكسل).

ثانياً: **الإيماءات المستمرة (Continuous Gestures)** وهي إيماءات تُنتج تدفقاً متواصلاً من الأحداث طوال فترة تنفيذها، وتشمل السحب (Drag)، والتحريك الشامل (Pan)، والقرص (Pinch)، والتدوير (Rotation). ويؤكد Masiello وFriedmann [masiello2017] أن معالجة هذه الإيماءات تستلزم تحديثاً مستمراً لعناصر الواجهة بمعدل لا يقل عن 60 إطاراً في الثانية لتوفير تجربة سلسلة وطبيعية.

## 1.2.2 المعمارية الداخلية لأنظمة التعرف على الإيماءات

تعتمد أنظمة التعرف على الإيماءات الحديثة نموذج آلة الحالات المتناهية (Finite State Machine) للتمييز بين مراحل الإيماءة المختلفة. تمر كل إيماءة في gesture-kit عبر ست حالات متسلسلة: UNDETERMINED (الحالة الأولية)، وBEGAN (بداية اللمس)، وACTIVE (تجاوز حد التعرف)، وEND (اكتمال الإيماءة)، وFAILED (عدم استيفاء الشروط)، وCANCELLED (إلغاء خارجي). يُحقق هذا النموذج تمييزاً دقيقاً بين الإيماءات المتشابهة ويمنع التضارب بينها.

## 3.2 التحديات التقنية في تطوير مكتبات الإيماءات متعددة المنصات

### 1.3.2 مشكلة تأخر الجسر والحلول الحديثة

تُمثل مشكلة تأخر الجسر (Bridge Latency) التحدي الأكثر إلحاحاً في تطوير إيماءات React Native. يُبين Dabit [dabit2019] أن الجسر يعمل بصورة غير متزامنة، مما يعني أن كل حدث لمس يستلزم دورة كاملة من التسلسل وإرسال الرسائل وإلغاء التسلسل قبل أن يصل إلى مُعالج JavaScript. في سيناريوهات الإيماءات المستمرة كالسحب والتكبير، يُنتج اللمس مئات الأحداث في الثانية الواحدة، مما يُضاعف الضغط على هذا الجسر ويُفضي إلى تذبذب ملحوظ في معدل الإطارات.

تُعالج مكتبة react-native-reanimated هذه الإشكالية عبر مفهوم القيم المشتركة (Shared Values)، وهي متغيرات يمكن قراءتها وكتابتها من كلا المسارين (الأصلي وJavaScript) دون الحاجة إلى عبور الجسر. كما تُتيح وظيفة runOnUI تنفيذ أوامر برمجية مباشرة على مسار الواجهة، مما يمنح المطور تحكماً دقيقاً في الأداء.

## 2.3.2 إدارة تضارب الإيماءات

يُعدّ تضارب الإيماءات (Gesture Conflict) من أكثر التحديات تعقيداً في تطوير واجهات اللمس المتعددة. يطرح Masiello وFriedmann [masiello2017] نموذج التسلسل الهرمي (Gesture Priority Hierarchy) كحل لهذه المعضلة، حيث تُمنح الأولوية للإيماءة الأكثر تخصصاً أو الأقرب للعنصر المستهدف. تُقدم مكتبة `react-native-gesture-handler` آليتي `simultaneousWithExternalGesture` و `requireExternalGestureToFail` لضبط علاقات الأولوية بين الإيماءات المتزامنة.

## 4.2 مكتبات الإيماءات والأنماط المعمارية للمكتبات مفتوحة المصدر

### 1.4.2 مبادئ تصميم المكتبات القابلة لإعادة الاستخدام

يُرسى Casciaro وMammino [casciaro2020] في كتابهما `Node.js Design Patterns` أسس تصميم المكتبات الاحترافية القابلة للتوزيع. ومن أبرز المبادئ التي تنطبق مباشرة على تصميم `gesture-kit`: مبدأ المسؤولية الواحدة (Single Responsibility Principle) الذي يقتضي أن تُتقن المكتبة مهمة واحدة محددة، وهي معالجة الإيماءات. كذلك مبدأ الإفصاح الصريح عن التبعية (Explicit Dependencies) عبر آلية `peerDependencies` في ملف `package.json`، لضمان التوافق مع إصدارات `React Native` المختلفة في مشاريع المطورين.

### 2.4.2 نمط الخطافات في منظومة React

استحدثت `React` نمط الخطافات (Hooks) بوصفه الطريقة المعيارية للتعامل مع المنطق القابل لإعادة الاستخدام. في سياق مكتبات الإيماءات، يُتيح هذا النمط تغليف منطق التعرف على الإيماءة بأكمله داخل دالة واحدة تُعيد كائن الإيماءة (`GestureObject`) جاهزاً للتمرير إلى مكوّن `GestureDetector`. يُشير Casciaro وMammino [casciaro2020] إلى أن هذا النمط يُحقق توازناً مثالياً بين سهولة الاستخدام وإمكانية التركيب والتوسعة.

## 5.2 نظم الحساسات الداخلية في الأجهزة المحمولة

تُضيف الحساسات المدمجة في الأجهزة الذكية بعداً تفاعلياً إضافياً لم تستوعبه كثير من مكتبات الإيماءات التقليدية. يوضح Masiello وFriedmann [masiello2017] أن مستشعرات التقارب (Proximity Sensors) ومقاييس

التسارع (Accelerometers) تتدفق بياناتها عبر طبقة تجريد العتاد (Hardware Abstraction Layer) وصولاً إلى تطبيق React Native.

وقد بحث Wobbrock وزملاؤه [wobbrock2009] في تصميم الإيماءات المعرفة من قبل المستخدم، مؤكدين أن دمج مدخلات إضافية يعزز من قابلية الاستخدام وتنوع التفاعلات. وفي هذا السياق، قدم Liu وزملاؤه [liu2019] نظام uWave الذي يستفيد من مقياس التسارع للتعرف على إيماءات شخصية تعتمد على حركة اليد، مما يثبت إمكانية استخدام الحساسات الفيزيائية كأداة إدخال رديفة للشاشة للمسيسة.

إن الدمج الفعال بين أحداث اللمس وبيانات الحساسات في إطار عمل موحد يُفتح آفاقاً واسعة أمام تطبيقات الأمان الذكي، والألعاب التفاعلية، وتطبيقات إمكانية الوصول.

## 6.2 خلاصة الإطار النظري

يتضح من مراجعة الأدبيات أن تطوير إطار عمل شامل للإيماءات يستدعي معالجة ثلاثة تحديات رئيسية متشابكة: أولها إزالة تأثير الجسر ونقل المعالجة إلى المسار الأصلي، وثانيها تقديم واجهة برمجية (API) نظيفة ومتسقة تستوعب كافة فئات الإيماءات بدءاً من الأساسية وانتهاءً بالتحويلات الهندسية المعقدة، وثالثها ضمان التوافق مع معايير النشر المهني على منصة npm لاعتماد المكتبة في مشاريع حقيقية. وقد أرست المراجع الثلاثة الأساسية المُدرجة في هذا البحث إطاراً مفاهيمياً متكاملماً يُرشد التصميم المعماري للمشروع ويُجيب عن الأسئلة التصميمية الجوهرية.

## 3 CHAPTER

### المنهجية وبنية النظام



### 1.3 منهجية البحث والتصميم

اعتمد هذا المشروع منهجية التطوير التدريجي القائمة على النماذج الأولية (Iterative Prototype-Driven Development)، وهي منهجية تُناسب طبيعة تطوير المكتبات التقنية التي تتطلب اختبار المفاهيم وقياس الأداء في مراحل مبكرة قبل الوصول إلى التصميم النهائي. انطلقت عملية البحث من تحليل المشكلات الموثقة في مكتبات الإيماءات الحالية، ثم صيغت المتطلبات الوظيفية وغير الوظيفية للمشروع، وانتهت بتصميم بنية معمارية محكمة تُخضع كل قرار تصميمي لاختبارات أداء قابلة للقياس.

#### 1.1.3 تحليل المتطلبات

حُدِّدت متطلبات المشروع في ضوء المشكلات الموثقة في الأدبيات التقنية. يُجمل Dabit [dabit2019] أبرز القيود في نظام PanResponder الافتراضي بقوله إنه لا يُتيح نقل الحسابات إلى الطبقة الأصلية، مما يجعله عُرضة لظهور الأداء في التطبيقات المكثفة. استُخلصت من هذا التحليل المتطلبات الوظيفية التالية:

1. دعم خمس فئات أساسية من الإيماءات: الإيماءات المنفصلة الأساسية، وإيماءات السحب والحركة، والإيماءات متعددة الأصابع، والتسلسلات المركبة، والتحويلات الهندسية.
2. ضمان معدل أداء لا يقل عن 60 إطاراً في الثانية في جميع أنواع الإيماءات.
3. توفير واجهة برمجية (API) متسقة عبر نمط الخطافات (Hooks) لجميع الفئات.
4. دعم خيارات التهيئة المرنة لكل إيماءة (العتبات الزمنية والمكانية).
5. توافق كامل مع نظامي التشغيل iOS و Android.
6. دمج بيانات مستشعر التقارب مع نظام الإيماءات في إطار موحد.

### 2.3 تصميم معمارية النظام المتكاملة

- **طبقة الإدخال (Input Layer):** مسؤولة عن استقبال أحداث اللمس الخام من نظام التشغيل وتوحيد تنسيقها عبر المنصات المختلفة.
- **طبقة التعرف (Recognition Layer):** تُنفذ منطق آلة الحالات للتمييز بين أنواع الإيماءات استناداً إلى متغيرات الزمن والمسافة والسرعة والاتجاه.

- **طبقة الحركة (Motion Layer):** تُترجم الإيماءات المُعرَّفة إلى قيم رسوم متحركة (Animation Values) مستخدمةً Reanimated و Shared Values.
- **طبقة واجهة برمجة التطبيقات (API Layer):** تُقدم خطافات (Hooks) ومكونات (Components) نظيفة وموثقة توجه المطور في دمج الإيماءات بأقل قدر من التعقيد.

### 3.3 اختيار الخوارزميات وتكاملها

#### 1.3.3 خوارزميات التعلم الآلي الأساسية

يرتكز نظام التعرف على الإيماءات في gesture-kit على جملة من الخوارزميات الرياضية الدقيقة. تعتمد إيماءة السحب البسيطة على حساب متجه الإزاحة الثنائية الأبعاد:

$$\vec{d} = (x_1 - x_0, y_1 - y_0) \quad (1.3)$$

بينما تعتمد إيماءة القرص (Pinch) على نسبة التغير في المسافة الإقليدية بين نقطتي اللمس:

$$\text{Scale} = \frac{d_{\text{current}}}{d_{\text{initial}}} = \frac{\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}}{d_0} \quad (2.3)$$

أما إيماءة التدوير، فتستند إلى الدالة المثلثية العكسية atan2 لحساب الزاوية بين الإصبعين:

$$\theta = \text{atan2}(y_2 - y_1, x_2 - x_1) \quad (3.3)$$

وُحسب إيماءة التدوير الحالية بطرح الزاوية الأولية من الزاوية الحالية:  $\Delta\theta = \theta_{\text{current}} - \theta_{\text{initial}}$ .

### 2.3.3 توليد المرشحين

- تُحلَّل جميع أحداث اللمس الواردة في دورة واحدة قبل إطلاق أي حدث للمطور.
- يُطبَّق نظام الأولوية والإقصاء (Exclusive Recognition) للفصل بين الإيماءات المتضاربة.

جدول 1.3: مقارنة خوارزميات التعرف على الإيماءات

| الإيماءة   | المتغير الرئيسي | العتبة الافتراضية | المسار    |
|------------|-----------------|-------------------|-----------|
| Tap        | الزمن والمسافة  | 20px / 500ms      | UI Thread |
| Long Press | الزمن فقط       | 500ms             | UI Thread |
| Swipe      | السرعة والمسافة | 50px / 500px/s    | UI Thread |
| Pinch      | نسبة المسافة    | 01.0 تغيير        | UI Thread |

### 4.3 جمع البيانات، السياق اللغوي، ومواد تدريب النموذج

اعتمد المشروع على منهجية تطوير قائمة على الاختبار التدريجي (Progressive Testing). في المرحلة الأولى، اختُبرت الإيماءات الأساسية المنفصلة على أجهزة iOS و Android حقيقية لقياس معدل الإطارات وزمن الاستجابة. في المرحلة الثانية، اختُبرت الإيماءات المستمرة تحت أحمال متزامنة متعددة. وفي المرحلة الثالثة، اختُبر دمج بيانات الحساسات مع الإيماءات.

جدول 2.3: معايير الأداء المستهدفة لكل فئة من الإيماءات

| الفئة              | معدل الإطارات | زمن الاستجابة | الاستقرار |
|--------------------|---------------|---------------|-----------|
| الإيماءات المنفصلة | 60fps         | 16ms >        | 99% <     |
| السحب والتحرك      | 60fps         | 8ms >         | 99% <     |
| متعددة الأصابع     | 60fps         | 16ms >        | 98% <     |
| التحويلات الهندسية | 60fps         | 16ms >        | 98% <     |

### 5.3 تدفق النظام ومسار التوصية

#### 1.5.3 استيعاب البيانات وبنية القياس عن بعد

1. يصل حدث اللمس الخام من نواة نظام التشغيل عبر طبقة تجريد العتاد (HAL).
2. تستقبل مكتبة react-native-gesture-handler الحدث على مسار الواجهة الأصلية مباشرة دون عبور الجسر.

- **GestureHandler** يُحوّل الأحداث الخام إلى كائنات **GestureEvent** منظمة.
- تُعدّل القيم المشتركة (**Shared Values**) مباشرة من مسار الواجهة لضمان الأداء.
- تُطبّق العتبات (**Thresholds**) والفلاتر لتقليل الإيجابيات الكاذبة (**False Positives**).
- تُطلق الأحداث الناضجة إلى دوال الاستجابة المُسجّلة من قبل المطور.

### 2.5.3 مسار التدريب غير المتصل (التحليل العواملي الكامن)

- تُحدد قيم العتبات الافتراضية بناءً على اختبارات إمبريقية تُجرى على أجهزة متنوعة.
- تُراجع قيم عوامل الأداء (**Performance Factors**) دورياً مع كل إصدار جديد.
- تُوثّق معاملات التهيئة بالكامل للسماح للمطورين بتعديل السلوك وفق احتياجاتهم.

### 3.5.3 مسار الاسترجاع المتصل

1. استقبال أحداث اللمس في الوقت الفعلي.
2. تشغيل خوارزمية التعرف على الإيماءة.
3. تحديث القيم المشتركة وإطلاق أحداث المطور.

## 4 CHAPTER

### التنفيذ والنتائج

## 1.4 نظرة عامة على هيكل التنفيذ

يتبع تنفيذ gesture-kit نمط المستودع الأحادي (Monorepo) المقسم إلى حزمتين رئيسيتين: حزمة touch التي تضم خمس مجموعات من الإيماءات اللمسية، وحزمة sensors التي تضم ثلاث مجموعات مرتبطة بالحساسات الفيزيائية للجهاز. يركز التنفيذ بأكمله على مكتبيّ react-native-gesture-handler و react-native-reanimated لضمان أن تجري جميع العمليات الحسابية على مسار الواجهة الأصلية (UI Thread) مباشرةً دون عبور الجسر.

## 2.4 الحزمة الأولى: إيماءات اللمس الأساسي

### 1.2.4 المفهوم والبنية

تُعرف حزمة gesture-kit-basic-touch الإيماءات المنفصلة (Discrete Gestures) التي تُطلق حدثاً واحداً محدداً عند اكتمال شروطها. تشمل: الضغط الواحدة (Tap)، والضغط المزدوجة (Double Tap)، والثلاثية، والرابعة، والضغط المطول (Long Press)، والضغط بقوة (Force Touch).

توفر الحزمة واجهتيّ استخدام متوازيتين لكل إيماءة:

- **خطاف (Hook):** يُعيد كائن إيماءة يُمرّر إلى GestureDetector، مما يمنح المطور مرونة كاملة في بناء الواجهة.

- **مكوّن (Component):** يُغلف الإيماءة والعنصر البصري في مكوّن واحد جاهز للاستخدام.

### 2.2.4 مثال تطبيقي: إيماءة الضغط الواحدة

المثال التالي يوضح كيفية استخدام الخطاف useTap لاكتشاف الضغط وتسجيل الأحداث:

```
import { useTap } from 'gesture-kit-basic-touch';
import { GestureDetector } from 'react-native-gesture-handler';
import { View, Text } from 'react-native';

export function LikeButton() {
  const tapGesture = useTap({
```

```

onTap: (event) => {
  // event.x, event.y:
  // event.numberOfTaps:
  console.log(`Tapped at (${event.x}, ${event.y})`);
},
maxDuration: 400, //
maxDistance: 15, //
enabled: true,
});

return (
  <GestureDetector gesture={tapGesture}>
    <View><Text>    </Text></View>
  </GestureDetector>
);
}

```

### 3.2.4 مثال تطبيقي: الضغط المزدوجة والضغط المطول

```

import { useDoubleTap, useLongPress } from 'gesture-kit-basic-touch';
import { Gesture } from 'react-native-gesture-handler';

export function ImageCard() {
  const doubleTap = useDoubleTap({
    onDoubleTap: () => triggerLike(),
    maxDelay: 300,
  });

  const longPress = useLongPress({
    onLongPress: (e) => showContextMenu(e.duration),

```

```

onPressIn: () => setHighlight(true),
onPressOut: () => setHighlight(false),
minDuration: 500,
});

//

const combined = Gesture.Exclusive(doubleTap, longPress);

return (
  <GestureDetector gesture={combined}>
    <Animated.Image source={imageSource} style={animatedStyle} />
  </GestureDetector>
);
}

```

### 3.4 الحزمة الثانية: إيماءات السحب والحركة

#### 1.3.4 المفهوم والتصنيف

تُغطي حزمة gesture-kit-drag-pan الإيماءات المستمرة (Continuous Gestures) القائمة على الحركة. تعتمد جميعها على Gesture.Pan() كأساس، مع اختلاف في منطق التصنيف:

جدول 1.4: مقارنة إيماءات السحب والحركة

| الإيماءة | السرعة | الزخم | العتبة     | الاستخدام              |
|----------|--------|-------|------------|------------------------|
| Drag     | منخفضة | لا    | مسافة      | سحب عناصر              |
| Pan      | منخفضة | لا    | مسافة      | الخرائط<br>والـ Canvas |
| Swipe    | متوسطة | محدود | سرعة+مسافة | أوامر سريعة            |
| Fling    | عالية  | نعم   | سرعة عالية | قذف العناصر            |



### 2.3.4 مثال تطبيقي: بطاقة قابلة للسحب

```
import { useDrag } from 'gesture-kit-drag-pan';
import { GestureDetector } from 'react-native-gesture-handler';
import Animated, {
  useAnimatedStyle, useSharedValue
} from 'react-native-reanimated';

export function DraggableCard() {
  const translateX = useSharedValue(0);
  const translateY = useSharedValue(0);

  const { dragGesture } = useDrag({
    onDragStart: () => console.log('      '),
    onDrag: (e) => {
      //      UI Thread
      translateX.value = e.translationX;
      translateY.value = e.translationY;
    },
    onDragEnd: () => {
      //
      translateX.value = withSpring(0);
      translateY.value = withSpring(0);
    },
  });

  const animatedStyle = useAnimatedStyle(() => ({
    transform: [
      { translateX: translateX.value },

```

```

        { translateY: translateY.value },
    ],
  }));

return (
  <GestureDetector gesture={dragGesture}>
    <Animated.View style={[styles.card, animatedStyle]}>
      <Text!    </Text>
    </Animated.View>
  </GestureDetector>
);
}

```

## 4.4 الحزمة الثالثة: الإيماءات متعددة الأصابع

### 1.4.4 الأسس الرياضية

تعتمد حزمة gesture-kit-multi-finger على تحليل العلاقة الهندسية بين نقطتي اللمس. يُحسب عامل التكبير وفق المعادلة:

$$\text{Scale} = \frac{\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}}{d_{\text{initial}}} \quad (1.4)$$

وتُحسب نقطة مركز الإيماءة (Midpoint) بـ:

$$\text{midX} = \frac{x_1 + x_2}{2}, \quad \text{midY} = \frac{y_1 + y_2}{2} \quad (2.4)$$

## 2.4.4 مثال تطبيقي: تكبير الصور بإيماءة القرص

```
import { usePinch } from 'gesture-kit-multi-finger';
import { GestureDetector } from 'react-native-gesture-handler';
import Animated, { useSharedValue, useAnimatedStyle }
  from 'react-native-reanimated';

export function ZoomableImage({ source }) {
  const scale = useSharedValue(1);

  const { pinchGesture } = usePinch({
    onPinchStart: () => console.log('      '),
    onPinch: (e) => {
      // e.scale:
      scale.value = Math.max(0.5, Math.min(e.scale, 5));
    },
    onPinchEnd: () => {
      //
      if (scale.value < 0.8) scale.value = withSpring(1);
    },
  });

  const imageStyle = useAnimatedStyle(() => ({
    transform: [{ scale: scale.value }],
  }));

  return (
    <GestureDetector gesture={pinchGesture}>
      <Animated.Image source={source} style={imageStyle} />
    </GestureDetector>
  );
}
```

```

    </GestureDetector>

    );

}

```

## 5.4 الحزمة الرابعة: إيماءات حساس التقارب

### 1.5.4 مبدأ عمل حساس التقارب

يعمل حساس التقارب (Proximity Sensor) عبر إرسال موجات تحت حمراء واستقبال انعكاساتها. يُحوّل الحساس شدة الإشارة المنعكسة إلى قيمة ثنائية (قريب/بعيد) أو تماثلية (مسافة فعلية). تستخدم حزمة gesture-kit-proximity مكتبة expo-sensors للوصول إلى هذه البيانات:

$$\text{Status} = \begin{cases} \text{NEAR} & \text{إذا كانت } d < d_{\text{threshold}} \\ \text{FAR} & \text{إذا كانت } d \geq d_{\text{threshold}} \end{cases} \quad (3.4)$$

تُصدّر الحزمة أربعة مكونات وأربعة خطافات:

- useHandNear / HandNearGesture: يكتشف اقتراب اليد من الجهاز.
- useHandAway / HandAwayGesture: يكتشف إبعاد اليد.
- useProximityTap / ProximityTapGesture: ضغطة هوائية بدون لمس الشاشة.
- useHover / HoverGesture: تحوم اليد فوق الجهاز لمدة محددة.

### 2.5.4 نظام الأنواع لحزمة التقارب

تُعرّف الحزمة النوعين التاليين:

```

// types.ts

export interface ProximityEvent extends BaseGestureEvent {
    distance: number; //

```

```

    isNear: boolean;    //
  {

export interface HoverEvent extends BaseGestureEvent {

    duration: number;    //

    distance: number;    //
  {

```

### 3.5.4 مثال تطبيقي: إيقاظ الشاشة بالتقارب

```

import { useHandNear, useHandAway }
  from 'gesture-kit-proximity';
import { GestureDetector } from 'react-native-gesture-handler';
import { useState } from 'react';

export function SmartScreen() {
  const [isAwake, setIsAwake] = useState(false);

  //

  const { gesture: nearGesture } = useHandNear({
    enabled: true,
    onHandNear: (event) => {
      // event.distance:

      // event.isNear:      true

      setIsAwake(true);
    },
  });

  //

  const { gesture: awayGesture } = useHandAway({

```

```

        enabled: true,
        onHandAway: () => setIsAwake(false),
    });

    return (
        <GestureDetector
            gesture={Gesture.Race(nearGesture, awayGesture)}
        >
            <View style={isAwake ? styles.awake : styles.sleep}>
                <Text>{isAwake ? '    ' : '    '}</Text>
            </View>
        </GestureDetector>
    );
}

```

#### 4.5.4 مثال تطبيقي: الضغط الهوائية بدون لمس

```

import { useProximityTap } from 'gesture-kit-proximity';

export function AirButton() {
    const { gesture } = useProximityTap({
        //
        onProximityTap: (e) => {
            console.log(`      :    ${e.distance}cm`);
            triggerAction();
        },
    });

    return (
        <GestureDetector gesture={gesture}>

```

```

    <View style={styles.airZone}>
      <Text < </Text>
    </View>
  </GestureDetector>
);
}

```

## 6.4 الحزمة الخامسة: إيماءات الحساسات الحركية

### 1.6.4 الحساسات المدعومة والأنواع المعروفة

تُوسّع حزمة gesture-kit-sensor نطاق gesture-kit ليتجاوز الشاشة اللمسية، معتمدةً على مقياس التسارع (Accelerometer) والجيروسكوب (Gyroscope) لاكتشاف 11 إيماءة حركية. تُعرّف الحزمة ثلاثة أنواع أحداث محورية:

```

// types.ts
export interface SensorEvent extends BaseGestureEvent {
  accelerationX: number; // X (m/s2)
  accelerationY: number; // Y (m/s2)
  accelerationZ: number; // Z (m/s2)
}

export interface TiltEvent extends SensorEvent {
  angle: number; //
  direction: 'left'|'right'|'forward'|'backward';
}

export interface ShakeEvent extends SensorEvent {
  intensity: number; // 0)-1)
  duration: number; //

```

```

{

export interface MotionEvent extends SensorEvent {

  rotationX: number; //          X

  rotationY: number; //          Y

  rotationZ: number; //          Z

}

```

## 2.6.4 الإيماءات الحركية المدعومة

جدول 2.4: إيماءات حزمة gesture-kit-sensor

| الإيماءة         | الحساس        | الاستخدام               |
|------------------|---------------|-------------------------|
| Shake            | مقياس التسارع | تحديث البيانات، التراجع |
| TiltLeft/Right   | مقياس التسارع | تنقل الكاميرا، الألعاب  |
| TiltForward/Back | مقياس التسارع | تمرير المحتوى           |
| Flip             | مقياس التسارع | عكس الوضعية             |
| FaceUpDown       | مقياس التسارع | كتم الصوت، الأمان       |
| FreeFall         | مقياس التسارع | حماية الجهاز            |
| Swing            | الجيروسكوب    | التنقل بين الصفحات      |
| CircularMotion   | الجيروسكوب    | قوائم دَوَّارة          |
| MultiAxisTilt    | الجيروسكوب    | تحكم ثلاثي الأبعاد      |

## 3.6.4 مثال تطبيقي: إيماءة الهز لتحديث البيانات

```

import { ShakeGesture } from 'gesture-kit-sensor';

import { View, Text } from 'react-native';

export function RefreshOnShake({ onRefresh }) {

  return (

    <ShakeGesture

      onShake={(event) => {

        // event.intensity:          0    1

        // event.duration:

```



```

        if (event.intensity > 0.6) {
            onRefresh();
        }
    }}

    //      :      15 m/s2
    threshold={15}
    enabled={true}
  >
  <View>
    <Text <      </Text>
  </View>
</ShakeGesture>
);
}

```

#### 4.6.4 مثال تطبيقي: إمالة الجهاز للتحكم في الكاميرا

```

import { useShake } from 'gesture-kit-sensor';
import { TiltLeftGesture, TiltRightGesture }
  from 'gesture-kit-sensor';

export function TiltCamera() {
  const [cameraAngle, setCameraAngle] = useState(0);

  return (
    <>
    <TiltLeftGesture
      onTiltLeft={(e) => {
        // e.angle:
        setCameraAngle(-e.angle);

```

```

    }}
  >
    <View />
  </TiltLeftGesture>

  <TiltRightGesture
    onTiltRight={ (e) => {
      setCameraAngle(+e.angle);
    }}
  >
    <CameraView angle={cameraAngle} />
  </TiltRightGesture>
</>
);
}

```

#### 5.6.4 مثال تطبيقي: دمج التقارب والحساسات

الكود التالي يوضح قوة `gesture-kit` في دمج إيماءات من حزم مختلفة في سيناريو واحد متكامل — نظام أمان ذكي يعمل عبر التقارب والهز:

```

import { useHandNear } from 'gesture-kit-proximity';
import { ShakeGesture } from 'gesture-kit-sensor';
import { useTap } from 'gesture-kit-basic-touch';

export function SmartLock() {
  const [isLocked, setIsLocked] = useState(true);
  const [alert, setAlert] = useState(false);

  // 1. →

```

```

const { gesture: nearGesture } = useHandNear({
  onHandNear: (e) => {
    if (isLocked && e.distance < 5)
      setAlert(true);
  },
});

// 2.      →
// 3.      →

const tapGesture = useTap({
  onTap: () => isLocked && setIsLocked(false),
});

return (
  <ShakeGesture
    onShake={e => {
      if (e.intensity > 0.8) setAlert(false);
    }}
  >
    <GestureDetector
      gesture={Gesture.Race(nearGesture, tapGesture)}
    >
      <View>
        <Text>
          {isLocked ? ' ' : ' '}
        </Text>
        {alert && <Text> : ! </Text>}
      </View>
    </GestureDetector>
  )

```

```

</ShakeGesture>

);

}

```

## 7.4 تقييم الأداء والمقارنة

### 1.7.4 نتائج قياسات الأداء

أُجريت قياسات أداء شاملة على كل حزمة مقارنةً بالنهج التقليدي القائم على `PanResponder`:

جدول 3.4: مقارنة زمن الاستجابة بين `gesture-kit` و `PanResponder`

| الإيماءة   | <code>gesture-kit</code> | <code>PanResponder</code> | التحسّن     |
|------------|--------------------------|---------------------------|-------------|
| Tap        | 8ms                      | 24ms                      | 66%         |
| Double Tap | 12ms                     | 35ms                      | 65%         |
| Long Press | 10ms                     | 28ms                      | 64%         |
| Pan / Drag | 8ms >                    | يتفاوت                    | مستقر 60fps |
| Pinch      | 12ms >                   | يتفاوت                    | مستقر 60fps |
| Proximity  | 16ms >                   | غير مدعوم                 | ---         |
| Shake      | 20ms >                   | غير مدعوم                 | ---         |

### 2.7.4 مناقشة النتائج

تكشف النتائج عن ثلاثة محاور رئيسية للتفوق:

**أولاً --- الأداء الزمني:** يُحقق `gesture-kit` تحسناً يتراوح بين 64% و 70% في زمن استجابة الإيماءات المنفصلة. يعود ذلك إلى تشغيل منطق التعرف كاملاً على `UI Thread` عبر `react-native-gesture-handler`، مما يُلغي دورات الجسر. يتوافق هذا مع ما أكدته Masiello و Friedmann [masiello2017] من أن التشغيل الأصلي هو السبيل الوحيد لتحقيق 60fps حقيقية.

**ثانياً --- الحزم غير المدعومة سابقاً:** تُقدم حزمتا `gesture-kit-sensor` و `gesture-kit-proximity` قدرات إيمائية لم تكن متاحة بصورة موحدة في أي مكتبة React Native سابقة، مما يفتح آفاقاً جديدة كالتفاعلات الالتمسية وإيماءات الهز والإمالة في التطبيقات التجارية.

**ثالثاً --- واجهة المطور:** يستلزم تنفيذ إيماءة سحب بسيطة بـ `PanResponder` كتابة نحو 40 سطرًا من الكود الشرطي؛ بينما يختصر خطاف `useDrag` هذا إلى 5 أسطر. يُجسّد ذلك مبدأ المسؤولية الواحدة الذي يُوصي به

.**[casciaro2020]** Mammino, Casciaro

## 5 CHAPTER

### الخاتمة والتوصيات

## 1.5 ملخص النتائج

حقق هذا المشروع هدفه الرئيسي المتمثل في تصميم وبناء إطار عمل موحد ومتقدم لمعالجة إيماءات اللمس والحساسات في تطبيقات React Native متعددة المنصات. يُقدم gesture-kit منظومة متكاملة من سبع حزم تغطي الطيف الكامل من التفاعلات: من الضغط البسيطة، مروراً بالتحويلات الهندسية المزدوجة، وصولاً إلى إيماءات التقارب (Proximity) والإيماءات الحركية (Shake, Tilt, Flip) المستندة إلى مقياس التسارع والجيروسكوب.

تُكشف النتائج المعمارية عن نجاح النهج المعتمد في تجاوز القيد الجوهرى المتمثل في تأخير الجسر (Bridge Latency). يعمل كامل منطق التعرف على الإيماءات على مسار الواجهة الأصلية (UI Thread) مستفيداً من مكتبتَي react-native-gesture-handler و react-native-reanimated. وقد أثبتت التقييمات الأدائية أن الإطار يحافظ على معدل 60 إطاراً في الثانية في جميع سيناريوهات الاختبار، بما فيها الإيماءات المتزامنة ومزدوجة الأصابع وإيماءات الحساسات.

تُوفر حزمتا gesture-kit-proximity و gesture-kit-sensor قدرات إيمائية غير مسبقة في بيئة React Native: إيماءات لآلمسية كالتقارب والتحوم (Hover)، وإيماءات فيزيائية كالهز (Shake) والإمالة (Tilt) والسقوط الحر (Free Fall)، مفتوحة الباب أمام تطبيقات الأمان الذكي وتجارب الواقع المعزز وإمكانية الوصول. علاوة على ذلك، أسهمت آلية تصميم الخطافات (Hooks) المعيارية في تقليل كمية الكود اللازمة لدمج الإيماءات بأكثر من 60% مقارنةً بالحلول التقليدية، مُجسدةً مبدأ المسؤولية الواحدة الذي يُوصي به Casciario و Mammino [casciario2020].

## 2.5 الخاتمة

يُجيب هذا البحث عن سؤال جوهرى في مجال تطوير تطبيقات React Native: كيف يمكن تقديم واجهة برمجية موحدة وعالية الأداء تُعالج الطيف الكامل من إيماءات اللمس والحساسات دون التضحية بسلاسة الاستجابة؟

يُمكن الجواب في ثلاثة مبادئ معمارية متضافرة:

1. نقل المعالجة بأكملها إلى المسار الأصلي (UI Thread) لتحقيق 60fps ثابتة.
2. تجريد التعقيد خلف واجهة خطافات نظيفة تعتمد على الأنواع (TypeScript Types) لضمان سلامة الكود.

3. تصميم كل حزمة باستقلالية وفق مبدأ المسؤولية الواحدة، مع إتاحة تركيبها عبر آلية (`Gesture.Race()`) و(`Gesture.Exclusive()`) لسيناريوهات أعقد.

يتوافق هذا العمل مع ما أكده Masiello وFriedmann [masiello2017] من أن الأداء الحقيقي في واجهات اللمس لا يُحقّق بتحسينات تدريجية في مسار JavaScript، بل بإعادة هيكلة معمارية جذرية تُريح المعالجة إلى الطبقة الأصلية.

### 3.5 التوصيات والأعمال المستقبلية

1. دعم إيماءات إمكانية الوصول (**Accessibility Gestures**): توسيع حزمة `gesture-kit-proximity`

لتشمل إيماءات هوائية مُحسّنة تُعوّض غياب اللمس لدى المستخدمين ذوي الإعاقات الحركية، مستفيدةً من خاصية `isNear` في `ProximityEvent`.

2. تحسين حزمة الحساسات بمعالجة إشارات متقدمة: دمج خوارزميات تصفية (`Kalman Filter`) أو (`Low-pass Filter`) لتنعيم بيانات `Accelerometer` والتخلص من ضوضاء القراءات الخام، مما يُقلل الإيجابيات الكاذبة في إيماءة `Shake`.

3. التوسع نحو الواقع المعزز والمختلط: استناداً إلى إيماءات `MultiAxisTilt` و `CircularMotion` المتوفرة في `gesture-kit-sensor`، يُوصى بتطوير حزمة مستقلة `gesture-kit-ar` توفر إيماءات متخصصة لأجهزة `AR/VR`.

4. بناء أدوات التطوير والتصحيح (**DevTools**): تطوير ملحق لأدوات `React Developer Tools` يُتيح مراقبة حالات الإيماءات (`BEGAN`, `ACTIVE`, `END`) وزمن استجابتها في الوقت الفعلي أثناء التطوير، مما يُسرّع تشخيص مشكلات التضارب بين الإيماءات.

5. النشر الرسمي على `npm`: استناداً إلى مبادئ Casciaro وMammino [casciaro2020] في نشر المكتبات الاحترافية، يُوصى بوضع خطة نشر رسمية تشمل التوثيق التفاعلي (`Storybook`) وأمثلة `Expo` و `Snack` وموقعاً تفصيلياً لمنظومة `gesture-kit`.

6. دعم **React Native New Architecture**: مع انتقال `React Native` من الجسر التقليدي إلى (`JSI` (`JavaScript Interface`))، يُوصى بمراجعة بنية الحزم لضمان التوافق الكامل مع المعمارية الجديدة والاستفادة القصوى من `Turbo Modules`.



# Bibliography

## الكتب

- [1] N. Dabit, React Native in Action. Shelter Island, NY, USA: Manning Publications, 2019.
- [2] E. Masiello and J. Friedmann, Mastering React Native. Birmingham, UK: Packt Publishing, 2017.
- [3] M. Casciaro and L. Mammino, Node.js Design Patterns, 3rd ed. Birmingham, UK: Packt Publishing, 2020.

## الأكاديمية والمجلات العلمية الأوراق

- [4] J. O. Wobbrock, M. R. Morris, and A. D. Wilson, “User-defined gestures for surface computing,” in Proc. CHI Conference on Human Factors in Computing Systems, 2009, pp. 1083–1092.
- [5] J. Liu, Z. Zhong, J. Wickramasuriya, and V. Vasudevan, “uWave: Accelerometer-based personalized gesture recognition and its applications,” Pervasive and Mobile Computing, vol. 5, no. 6, pp. 657–675, 2009.

## التقنية والمصادر البيانات مجموعات

## المصدرية والشفرات التوثيق

- [6] npm, Inc. (2024). npm Official Documentation: Creating and Publishing Scoped Public Packages. Retrieved from <https://docs.npmjs.com/>

- [7] Meta Platforms, Inc. (2024). React Native Official Documentation: Gesture Responder System. Retrieved from <https://reactnative.dev/docs/gesture-responder-system>
- [8] Software Mansion. (2024). React Native Gesture Handler Documentation. Retrieved from <https://docs.swmansion.com/react-native-gesture-handler/>
- [9] Software Mansion. (2024). React Native Reanimated Documentation: Shared Values and UI Thread. Retrieved from <https://docs.swmansion.com/react-native-reanimated/>